NEXUS Language Manual

Version 2.0.0
Author: fanxcz
Repository: https://github.com/fanxcz/nexus-lang

---

1. Introduction

NEXUS is a compiled programming language designed for native applications, tools, and games. The reference compiler is written in C++20. Programs are compiled to native code through LLVM IR or through a small built-in native backend.

The language is intentionally small, explicit, and predictable. It does not hide control flow or memory operations. Variables are stack slots, function calls are direct, and the generated LLVM IR can be inspected and compiled with a normal C++ toolchain.

NEXUS has three major parts:

1. **Compiler** (src/) – lexer, parser, semantic analysis, and code generation.
2. **Runtime** (src/runtime/runtime.cpp) – console, files, networking, graphics, audio, input, and helper functions.
3. **Standard library** (stdlib/) – convenience wrappers written in NEXUS.
4. **Self-hosting toolchain** (selfhost/) – lexer, parser, type checker, and code generator written in NEXUS.

The LLVM backend is the recommended backend. It supports the full language. The native backend

supports only a small subset and exists for experiments and tiny programs.

---

2. Compilation model

A NEXUS program is compiled in several stages:

1. **Lexical analysis** converts source text into tokens.
2. **Parsing** builds an abstract syntax tree.
3. **Semantic analysis** checks types and variable usage.
4. **Code generation** emits LLVM IR or native code.
5. **External compilation** (for LLVM) compiles .ll files with a clang-compatible driver and links the runtime.

This separation makes the compiler easier to debug. The generated .ll file is ordinary text and can be inspected directly.

---

3. Installation and build

3.1 Building from source

```
cmake -S . -B build
cmake --build build -j
ctest --test-dir build
```

The compiler is produced as build/nexus (or build/nexus.exe on Windows).

3.2 Environment variables

- NEXUS_CXX – clang-compatible driver used by self-hosting tests and by generated program compilation.

- NEXUS_SOURCE_ROOT — source root used by the compiler (set by CMake).

3.3 Self-hosting environment

For self-hosting tests on Windows, NEXUS_CXX can point to build/clang++.cmd, which wraps the bundled Zig toolchain.

---

4. First program

Create hello.nx:

```
fn main() {
    print("Hello, NEXUS!")
}
```

Run it:

```
nexus run hello.nx --backend llvm
```

What happens:

1. The compiler reads hello.nx.
2. It generates LLVM IR.
3. The runtime is linked.
4. The resulting program prints the message.

---

5. Lexical structure

5.1 Comments

Line comments begin with // and continue to the end of the line:

```
let x = 10 // comment
```

Block comments use /* and */:

```
/* This is
   a block comment. */
```

5.2 Identifiers

Identifiers consist of letters, digits, and underscores. They must not begin with a digit. Identifiers are case-sensitive.

5.3 Keywords

Keywords are reserved and cannot be used as identifiers:

```
fn let const mut return if else while for in break
struct enum import extern unsafe as match true fals
```

5.4 Literals

Integer literals are written in decimal form:

```
0
42
-7
```

Floating point literals contain a decimal point:

```
3.14
1.0
0.5
```

String literals are enclosed in double quotes:

```
"hello"
```

Boolean literals are:

```
true
false
```

---

6. Types

NEXUS has a small, explicit type system. Every value has a type that is known before the program runs. Function parameters and return types are always written explicitly; local variables may be inferred from their initializer.

6.1 Primitive types

Type	Meaning
i64	signed 64-bit integer
f64	64-bit floating point number
bool	boolean value
string	text
void	no value

int, isize, and usize are accepted as aliases for i64. float is an alias for f64.

Type annotations are part of a function declaration:

```
fn scale(x: f64, factor: f64) -> f64 {
    return x * factor
}
```

A local variable can state its type explicitly:

```
let count: i64 = 10
let name: string = "Nexus"
```

6.2 Struct types

A struct groups named fields. Each field has its own type:

```
struct Point {
    x: i64,
    y: i64
}
```

Create a value by listing every field:

```
let p = Point { x: 3, y: 4 }
print(p.x + p.y)
```

Fields are accessed with a dot. A mutable binding allows field assignment:

```
let mut p = Point { x: 3, y: 4 }
p.x = p.x + 10
print(p.x)
```

The initializer must provide every field exactly once, and each value must match the field type. Structs may contain other structs:

```
struct Line {
    start: Point,
    end: Point
}

let line = Line {
    start: Point { x: 0, y: 0 },
    end: Point { x: 10, y: 10 }
```

```
}
print(line.end.x)
```

Structs are value types: assigning one struct variable to another copies the value. They are not nullable and there is no inheritance.

6.3 Enum types

An enum is a named set of variants:

```
enum Color {
    Red,
    Green,
    Blue
}
```

A variant is written with `::`:

```
let color = Color::Green
```

Enums are compared with `==` and `≠` and are usually consumed with `match`:

```
match color {
    Color::Red ⇒ { print("red") }
    Color::Green ⇒ { print("green") }
    Color::Blue ⇒ { print("blue") }
}
```

Enum values may be passed to functions, returned from functions, stored in struct fields, and stored in arrays:

```
fn color_name(color: Color) → string {
    match color {
        Color::Red ⇒ { return "red" }
        Color::Green ⇒ { return "green" }
```

```
        Color::Blue ⇒ { return "blue" }
        _ ⇒ { return "unknown" }
    }
}
```

Variants do not carry payload data in the current language. Enum values have an integer representation, but the compiler does not expose it; use match rather than relying on numeric values.

6.4 Array types

An array is a fixed-size, homogeneous sequence:

```
let numbers = [10, 20, 30]
print(numbers[0])
```

The type of numbers is [i64; 3]. The size may also be written explicitly:

```
let names: [string; 2] = ["Ada", "Grace"]
```

Indexes start at zero and must be i64. Constant indexes are checked at compile time; dynamic indexes are checked at run time and a failed check stops the program with a runtime error instead of reading invalid memory.

Arrays are mutable only through a mutable binding:

```
let mut values = [1, 2, 3]
values[1] = 42
print(values[0] + values[1])
```

The element type is inferred from the first element; every other element must have the same type. Empty array literals are not supported. Arrays are value types, so assigning an array copies its elements.

6.5 Pointer types

A pointer stores the address of a value. Take an address with & and read the pointee with *:

```
let value = 7
let pointer = &value
print(*pointer)
```

The type of pointer is *i64. Address-of works for variables, struct fields, and array elements. A pointer type is written with * before the element type:

```
let pointer: *i64 = &value
```

Pointers are primarily useful for external functions. NEXUS does not have pointer arithmetic or manual memory management in the language; memory for strings, lists, and runtime objects is managed by the runtime.

6.6 Type compatibility

- i64 and f64 are different types. There is no implicit conversion between them; use to_f64 and to_i64 from the runtime.
- bool is not an integer. It cannot be used in arithmetic.
- Two struct types are compatible only when they have the same name.
- Two enum types are compatible only when they have the same name.
- Array types are compatible when their element types and lengths match.
- String concatenation with + accepts a string on either side and converts an i64, f64, or bool operand to text automatically:

nx let message = "count = " + 42

- A let without an annotation takes the type of its initializer. A let with an annotation requires the initializer to have exactly that type.

---

7. Variables

7.1 Immutable variables

Use let to declare a variable whose value does not change:

```
let x = 10
```

After declaration, x cannot be reassigned.

7.2 Mutable variables

Use let mut when the value will change:

```
let mut i = 0
i = i + 1
```

7.3 Constants

Use const for a binding that is declared once and never reassigned:

```
const limit: i64 = 10
const greeting = "hello"
```

const requires an initializer and, like let, produces an immutable binding. An explicit type is optional.

7.4 Discarding values

Use let _ to explicitly discard a value:

```
let _ = some_function()
```

This is useful when a function returns a value that is not needed. Each _ is independent, so let _ may be repeated in the same block.

7.5 Type stability

A variable keeps its type for its entire lifetime. Reassigning a value of a different type is an error:

```
let mut x = 10
x = 20      // ok
x = "text"  // error
```

A mutable binding is required for any assignment. let x = 10; x = 20 is an error because x is immutable.

7.6 Scope

Variables are scoped to the block in which they are declared. A variable declared inside an if, while, for, or match block is not visible outside that block. An inner block may declare a new variable with the same name as an outer one; the inner declaration shadows the outer one only inside that block. Two variables with the same name in the same block are an error.

---

8. Expressions

8.1 Arithmetic

```
a + b
a - b
a * b
a / b
a % b
```

Integer operations use signed 64-bit arithmetic. If either operand is f64, the operation is performed in floating point.

8.2 Comparisons

```
a == b
a ≠ b
a < b
a ≤ b
a > b
a ≥ b
```

Comparisons produce a bool.

8.3 Logical operators

```
a && b
a || b
!a
```

&& and || operate on boolean values.

8.4 Unary minus

```
-x
```

For i64, this computes zero minus the value. For f64, it negates the floating point value.

8.5 String concatenation

```
let s = "hello" + " " + "world"
```

The + operator concatenates strings when at least one operand is a string.

8.6 Precedence

From highest to lowest:

1. Primary expressions and function calls
2. Unary - and !
3. *, /, %
4. +, -
5. Comparisons
6. &&
7. ||

Parentheses can be used to override precedence.

---

9. Control flow

9.1 If statement

```
if condition {
    ...
} else {
    ...
}
```

The condition must be boolean. The else branch is optional.

9.2 While loop

```
while condition {
    ...
}
```

The condition is evaluated before each iteration.

9.3 For loop

A numeric range loop runs from a start value up to, but not including, an end value:

```
for i in 0..5 {
    print(i)
}
```

The loop variable is declared by the loop and is visible only inside the loop body. Like let, it is immutable. The bounds are evaluated once, before the loop starts, and must be i64.

A second form iterates over the elements of an array:

```
let names = ["Ada", "Grace", "Linus"]
for name in names {
    print(name)
}
```

The array expression is evaluated once. Each element is copied into the loop variable in order, so the loop variable has the element type. Modifying the array inside the loop does not change which elements the loop visits.

break leaves the loop and continue starts the next iteration. Both work in range and array loops:

```
for i in 0..10 {
    if i == 3 {
        continue
    }
    if i == 6 {
        break
    }
    print(i)
}
```

Iteration over strings is not supported; use string.len, string.byte_at, or string.slice with a numeric range instead.

9.4 Break and continue

Inside a loop, break leaves the loop immediately and continue jumps to the next iteration:

```
let mut i = 0
while i < 10 {
    i = i + 1
    if i == 3 {
        continue
    }
    if i == 6 {
        break
    }
    print(i)
}
```

The same statements work inside for loops and inside a while loop that was generated from a for loop.

9.5 Pattern matching

match compares a value against a list of arms. The first arm that matches is executed. Each arm has a pattern, an optional guard, an arrow, and a block:

```
let status = 2
match status {
    0 ⇒ { print("idle") }
    1 ⇒ { print("starting") }
    _ ⇒ { print("running") }
}
```

The value may be an i64, a bool, or an enum value.

Literal patterns

Integer, boolean, and enum patterns match by equality:

```
match color {
    Color::Red ⇒ { print("red") }
    Color::Green ⇒ { print("green") }
    _ ⇒ { print("other") }
}
```

Two identical literal patterns in the same match are an error.

Range patterns

Integer ranges use the same start..end syntax as for loops. Both ends are inclusive:

```
match age {
    0..12 ⇒ { print("child") }
    13..17 ⇒ { print("teenager") }
    18..64 ⇒ { print("adult") }
    _ ⇒ { print("senior") }
}
```

Range patterns require an i64 value. The lower bound must not be greater than the upper bound when both are literals.

Binding patterns

A bare identifier followed by ⇒ or by a guard binds the matched value to a local variable:

```
match score {
    100 ⇒ { print("perfect") }
    value if value > 90 ⇒ { print("great: " + str_
    value ⇒ { print("scored " + str_i64(value)) }
}
```

A binding is local to its arm. It does not modify or shadow the matched expression outside the arm.

Guards

An arm may add `if` condition after the pattern. The arm runs only when the pattern matches **and** the guard is true. Guards may use bound values and any variables from the surrounding scope:

```
let limit = 10
match n {
    value if value < limit ⇒ { print("below limit"
    _ ⇒ { print("at or above limit") }
}
```

A guard must have type `bool`.

Catch-all and reachability

`_` is the wildcard pattern and matches every value. A binding without a guard also matches every value. Either form is a catch-all, so it must be the last arm:

```
match value {
    1 ⇒ { print("one") }
```

```
    _ ⇒ { print("other") }
}
```

The compiler rejects arms that appear after a catch-all because they would be unreachable. A match does not have to be exhaustive: if no arm matches, the match simply does nothing and execution continues after it. To make a function return on every path, include a catch-all arm or add a return after the match.

Matching booleans

```
match ready {
    true ⇒ { print("ready") }
    false ⇒ { print("waiting") }
}
```

9.6 Blocks

Blocks are delimited by braces. They introduce a new scope:

```
{
    let x = 1
    print(x)
}
```

---

10. Functions

10.1 Declaration

```
fn name(param: type, ...) → type {
    ...
}
```

10.2 Parameters

Parameters are named and typed:

```
fn add(a: i64, b: i64) → i64 {
    return a + b
}
```

10.3 Return

```
return value
```

A function may return early. A non-void function must return a value on every reachable path; otherwise the compiler reports an error. main may omit a return and exits with status 0.

10.4 Main function

```
fn main() {
    print("Hello")
}
```

main is the program entry point. At the LLVM level it returns i32.

10.5 Function calls

```
let x = add(1, 2)
print(add(x, 3))
```

10.6 Recursion

Functions may call themselves:

```
fn factorial(n: i64) → i64 {
    if n ≤ 1 {
        return 1
    }
    return n * factorial(n - 1)
}
```

10.7 Function order

Functions must be declared before they are called. The compiler processes declarations in source order.

10.8 Void functions

A function without → type returns void:

```
fn greet(name: string) {
    print("hello, " + name)
}
```

A void function may use return without a value to exit early, but it cannot return a value.

10.9 External functions

extern "C" fn declares a function that is provided outside the NEXUS program, usually by the runtime or the operating system:

```
extern "C" fn strlen(text: string) → i64

fn main() {
    print(strlen("hello"))
}
```

The string after extern names the calling convention; "C" is the supported convention. A declaration has no body and ends with ; (the

semicolon is optional because NEXUS generally allows omitted semicolons). string maps to a C string pointer and the primitive numeric types map to their C equivalents. The symbol must be available when the generated program is linked.

---

11. Strings

11.1 Literals

```
let s = "hello"
```

11.2 Escape sequences

```
"\n"   // newline
"\t"   // tab
"\r"   // carriage return
"\\"   // backslash
"\""   // double quote
```

11.3 Concatenation

```
let s = "a" + " " + "b"
```

11.4 Comparison

Strings can be compared for equality and inequality:

```
print("nexus" == "nexus")
print("a" ≠ "b")
```

Ordering comparisons (<, ≤, >, ≥) are not supported for strings.

11.5 Printing strings

```
print("hello")
```

print writes the string followed by a newline. A string expression can be built from text and numbers with +:

```
print("answer = " + 42)
```

11.6 UTF-8 and byte operations

Source files and string literals are UTF-8. The compiler stores a string as a sequence of UTF-8 bytes and prints those bytes unchanged, so non-ASCII text works as long as the terminal or output target interprets UTF-8:

```
print("Привет, мир")
```

The string functions in the standard library are byte-oriented:

- string.len returns the number of bytes, not the number of Unicode characters.
- string.byte_at returns one byte (0–255).
- string.slice uses byte offsets.
- string.upper and string.lower change ASCII letters only.

There are no \u escape sequences in the current language. Write non-ASCII text directly in the source file as UTF-8.

---

12. Floating point

Floating point values use the f64 type:

```
let x = 1.5
let y = 2.0
print(x + y)
print(x * y)
print(x / y)
```

Comparisons:

```
if x < y {
    print("less")
}
```

Floating point arithmetic follows IEEE-754 semantics through LLVM.

---

13. Modules and imports

Import a module by file name:

```
import "io.nx"
```

The standard library is located in stdlib/. Imports make the module's functions available.

---

14. Standard library

14.1 io

```
read_line(prompt: string) → string
read_i64(prompt: string) → i64
read_f64(prompt: string) → f64
```

14.2 fs

```
fs_read(path: string) → string
fs_write(path: string, data: string) → i64
fs_exists(path: string) → bool
```

14.3 string

```
len(s: string) → i64
byte_at(s: string, index: i64) → i64
byte_len(s: string) → i64
```

14.4 os

```
args_count() → i64
args_get(index: i64) → string
```

14.5 net

```
http_get_text(url: string) → string
```

14.6 json

```
json_field(json: string, key: string) → string
```

14.7 db

```
db_open(path: string) → i64
db_exec(db: i64, sql: string) → i64
db_close(db: i64)
```

14.8 crypto

```
hash_sha256(text: string) → string
```

14.9 memory

```
memory_alloc(bytes: i64) → i64
memory_free(ptr: i64)
```

---

15. Runtime functions

The runtime provides low-level functions used by generated code:

```
nexus_print_i64(i64)
nexus_print_f64(double)
nexus_print_bool(i1)
nexus_print_str(ptr)

nexus_str_concat(ptr, ptr) → ptr
nexus_str_equal(ptr, ptr) → i1

nexus_file_read(ptr) → ptr
nexus_file_write(ptr, ptr) → i64
nexus_file_exists(ptr) → i1

nexus_env(ptr) → ptr
nexus_system(ptr) → i64
nexus_exit(i64)
```

These functions are declared automatically in generated LLVM IR.

---

16. Tooling

16.1 Running programs

```
nexus run file.nx --backend llvm
```

16.2 Type checking

```
nexus check file.nx
```

16.3 Building

```
nexus build
```

16.4 REPL

```
nexus repl
```

16.5 Formatting

```
nexus fmt file.nx
```

16.6 Self-test

```
nexus self-test
```

16.7 Doctor

```
nexus doctor
```

16.8 Backends

- llvm supports the full language.
- native supports only a small subset.

Use --backend llvm for all real programs.

---

17. Build system

NEXUS uses CMake:

```
cmake -S . -B build
cmake --build build
ctest --test-dir build
```

The build produces the compiler, tests, and runtime tests.

---

18. Native backend

The native backend is intentionally small. It is useful for experiments but does not support strings, functions with full type support, or most standard library features.

For any non-trivial program, use:

```
nexus run file.nx --backend llvm
```

---

19. Self-hosting

The selfhost/ directory contains a compiler written in NEXUS:

- lexer_lib.nx
- parser_lib.nx
- typecheck_lib.nx

- codegen_lib.nx

The self-hosted code generator supports:

- let and let mut
- assignment
- print
- arithmetic
- comparisons
- logical operators
- unary minus
- if and else
- while
- for i in start..end
- break and continue
- const, let, and let mut
- functions and calls
- string literals
- f64
- booleans and logical negation (!)

Self-hosting is considered complete only when the self-hosted compiler produces output matching the C++ compiler on the test corpus.

---

20. Grammar

```
program      := declaration*
declaration  := import | struct | enum | extern | f
import       := "import" (STRING | IDENT ("." IDENT
struct       := "struct" IDENT "{" field* "}"
field        := IDENT ":" type ("," | ";")?
enum         := "enum" IDENT "{" IDENT ("," | ";")*
extern       := "extern" STRING function_signature
function     := "fn" IDENT "(" params ")" ("→" typ
function_signature := IDENT "(" params ")" ("→" ty
params       := (param ("," param)*)?
param        := IDENT ":" type
type         := "i64" | "f64" | "bool" | "string" |
```

```
              | "*" type
              | "[" type ";" INTEGER "]"
              | IDENT

block         := "{" statement* "}"
statement     := let | const | assign | print | if |
              | break | continue | return | block |
let           := "let" "mut"? IDENT (":" type)? "="
const         := "const" IDENT (":" type)? "=" expre
assign        := lvalue ("=" | "+=" | "-=" | "*=" |
lvalue        := IDENT | expression "." IDENT | expr
print         := "print" "(" expression ")"
if            := "if" expression block ("else" (bloc
while         := "while" expression block
for           := "for" IDENT "in" (expression ".." e
match         := "match" expression "{" arm* "}"
arm           := pattern ("if" expression)? "⇒" blo
pattern       := expression (".." expression)? | IDE
break         := "break"
continue      := "continue"
return        := "return" expression?

expression    := binary expression with precedence:
                 ||  <  &&  <  comparison  <  + -  <
unary         := ("-" | "!" | "&" | "*") unary | pos
postfix       := primary ("(" arguments ")")? ("." I
primary       := INTEGER | FLOAT | STRING | "true" |
              | IDENT | IDENT "::" IDENT
              | IDENT "{" field_initializer ("," fi
              | "[" expression ("," expression)* "]
              | "(" expression ")"
arguments     := (expression ("," expression)*)?
```

---

21. Examples

21.1 Factorial

```
fn factorial(n: i64) → i64 {
    if n ≤ 1 {
        return 1
    }
    return n * factorial(n - 1)
}

fn main() {
    print(factorial(10))
}
```

21.2 Floating point

```
fn main() {
    let x = 1.5
    let y = 2.0
    print(x + y)
    print(x * y)
}
```

21.3 Strings

```
fn main() {
    print("hello")
    print("a" + " " + "b")
}
```

21.4 Loop and sum

```
fn main() {
    let mut i = 0
    let mut sum = 0
```

```
    while i < 5 {
        sum = sum + i
        i = i + 1
    }
    print(sum)
}
```

21.5 Functions

```
fn mul(a: i64, b: i64) → i64 {
    return a * b
}

fn main() {
    print(mul(6, 7))
}
```

21.6 Structs, enums, arrays, and match

```
struct Task {
    title: string,
    priority: i64
}

enum State {
    Todo,
    Doing,
    Done
}

fn state_name(state: State) → string {
    match state {
        State::Todo ⇒ { return "todo" }
        State::Doing ⇒ { return "doing" }
        State::Done ⇒ { return "done" }
        _ ⇒ { return "unknown" }
    }
}
```

```
fn priority_label(priority: i64) -> string {
    match priority {
        0..3 => { return "low" }
        4..7 => { return "normal" }
        value if value > 7 => { return "urgent(" +
        _ => { return "unset" }
    }
}

fn main() {
    let mut tasks = [
        Task { title: "write docs", priority: 3 },
        Task { title: "ship it", priority: 9 },
        Task { title: "review", priority: 7 }
    ]
    tasks[0].priority = 5

    let states = [State::Todo, State::Doing, State:
    for i in 0..3 {
        print(tasks[i].title + ": " + state_name(st
              priority_label(tasks[i].priority))
    }
}
```

This example writes write docs: todo, normal, ship it: doing, urgent(9), and review: done, normal.

---

22. Troubleshooting

Program does not run

Use the LLVM backend:

```
nexus run file.nx --backend llvm
```

Link errors

Ensure NEXUS_CXX points to a clang-compatible driver.

Type errors

Run:

```
nexus check file.nx
```

Self-hosting tests fail

Ensure a clang-compatible compiler is available and NEXUS_CXX is set.

---

23. Design notes

NEXUS favors explicitness over implicit behavior. Types are stated in function signatures, variables must be declared before use, and the generated LLVM IR is readable. This makes the language suitable for learning, tooling, and systems-oriented programming.

---

24. Notes

- Functions must be declared before use.
- Use let mut for mutable variables.
- let _ = ... discards a value.
- Native backend is limited; use LLVM backend for full programs.
- Documentation is versioned with the repository.

---

25. Language features and status

This section records what the current compiler implements and what is still planned. A feature is

listed as implemented only when it is covered by the test suite or by a documented example in this manual.

25.1 Implemented

Feature	Notes
Primitive types	i64, f64, bool, string, void
Structs	named fields, nesting, field assignment
Enums	named variants, equality, match, function parameters and results
Arrays	fixed size, indexing, assignment, runtime bounds checks
Pointers	address-of, dereference, pointer types
Immutable and mutable bindings	let, let mut, const
Local type inference	inferred from the initializer, explicit annotations checked
Functions	parameters, return values, recursion, void functions
External functions	extern "C" fn declarations
Imports	import "file.nx" and dotted module names
Control flow	if/else, while, for over ranges and arrays, break, continue
Pattern matching	literals, enums, booleans, ranges, bindings, guards
Strings	

Feature	Notes
	concatenation, equality, UTF-8 pass-through, byte operations
Floating point	IEEE-754 arithmetic and comparisons
Diagnostics	NX1001 syntax, NX2000/NX2001 semantic errors with locations
LSP	hover, completion, definition, references, rename, symbols, formatting
Self-hosted compiler	lexer, parser, type checker, and LLVM IR subset

25.2 Planned, not implemented yet

The following features are not part of the current language. They are listed so that documentation and examples do not imply support that does not exist.

Feature	Current status
Generic functions and types	not implemented
Traits or interfaces	not implemented
Async/await and concurrency	not implemented
Lambdas and closures	not implemented
Function overloading	not implemented; function names must be unique
Option<T> / Result<T, E>	not implemented; enums have no payloads
Enum payloads and destructuring	not implemented

Feature	Current status
match exhaustiveness checking	not implemented; unmatched values fall through
Iterators and lazy sequences	not implemented
Macros, attributes, #[…]	not implemented
Compile-time evaluation of const	not implemented; const is an immutable binding
Incremental compilation cache	not implemented
LLVM optimization passes	not implemented; the LLVM backend emits textual IR
Full native backend parity	partial; the native backend supports a small subset
Full self-hosting parity	partial; the self-hosted compiler covers a subset
Unicode-aware string indexing	not implemented; string APIs are byte-oriented
\u escape sequences	not implemented; write UTF-8 directly in source
Doc comments and generated API docs	not implemented

25.3 Compatibility promise

The implemented features in section 25.1 are exercised by the repository test suite. Planned features may change before release. Programs that need a stable target should use the LLVM backend and the features listed as implemented.

---

26. Runtime function reference

This appendix lists the runtime functions available to generated programs.

Animation

Function
nexus_gfx_anim_create
nexus_gfx_anim_destroy
nexus_gfx_anim_draw
nexus_gfx_anim_frame
nexus_gfx_anim_update

Audio

Function
nexus_gfx_music_load
nexus_gfx_music_play
nexus_gfx_music_stop
nexus_gfx_music_volume
nexus_gfx_sound_load
nexus_gfx_sound_play
nexus_gfx_sound_playing
nexus_gfx_sound_stop
nexus_gfx_sound_tone
nexus_gfx_sound_unload
nexus_gfx_sound_volume

Canvas

Function
nexus_gfx_canvas_brush
nexus_gfx_canvas_checkpoint
nexus_gfx_canvas_clear
nexus_gfx_canvas_create
nexus_gfx_canvas_destroy
nexus_gfx_canvas_draw
nexus_gfx_canvas_fill
nexus_gfx_canvas_height
nexus_gfx_canvas_line
nexus_gfx_canvas_load_bmp
nexus_gfx_canvas_rect
nexus_gfx_canvas_redo
nexus_gfx_canvas_save_bmp
nexus_gfx_canvas_undo
nexus_gfx_canvas_width

Crypto

Function
nexus_sha256

Database

Function
nexus_sqlite_close

Function
nexus_sqlite_exec
nexus_sqlite_open

ECS

Function
nexus_ecs_alive
nexus_ecs_collides
nexus_ecs_count
nexus_ecs_create
nexus_ecs_destroy
nexus_ecs_draw
nexus_ecs_set_color
nexus_ecs_set_position
nexus_ecs_set_size
nexus_ecs_set_texture
nexus_ecs_set_velocity
nexus_ecs_update
nexus_ecs_x
nexus_ecs_y

Files

Function
nexus_file_exists
nexus_file_read
nexus_file_write

Graphics 2D

Function
nexus_gfx_audio_available
nexus_gfx_audio_error
nexus_gfx_backend
nexus_gfx_begin
nexus_gfx_camera_reset
nexus_gfx_camera_set
nexus_gfx_circle
nexus_gfx_clear
nexus_gfx_cursor_visible
nexus_gfx_dt
nexus_gfx_end
nexus_gfx_error
nexus_gfx_fullscreen
nexus_gfx_height
nexus_gfx_image_type
nexus_gfx_init
nexus_gfx_is_fullscreen
nexus_gfx_line
nexus_gfx_maximize
nexus_gfx_poll
nexus_gfx_rect
nexus_gfx_restore
nexus_gfx_set_title

Function
nexus_gfx_set_ui_resolution
nexus_gfx_should_close
nexus_gfx_shutdown
nexus_gfx_vsync
nexus_gfx_width

Graphics 3D

Function
nexus_gfx3d_begin
nexus_gfx3d_cube
nexus_gfx3d_end
nexus_gfx3d_grid
nexus_gfx3d_model_draw
nexus_gfx3d_model_load
nexus_gfx3d_model_unload

Input

Function
nexus_gfx_event_is
nexus_gfx_event_key
nexus_gfx_event_mouse_button
nexus_gfx_event_type
nexus_gfx_event_x
nexus_gfx_event_y
nexus_gfx_key_down

Function
nexus_gfx_mouse_down
nexus_gfx_mouse_x
nexus_gfx_mouse_y
nexus_input
nexus_input_f64
nexus_input_i64

JSON

Function
nexus_json_get

Lists

Function
nexus_ilist_get
nexus_ilist_len
nexus_ilist_new
nexus_ilist_push
nexus_ilist_set
nexus_slist_get
nexus_slist_len
nexus_slist_new
nexus_slist_push

Math

Function
nexus_sqrt
nexus_to_f64
nexus_to_i64

Memory

Function
nexus_mem_alloc
nexus_mem_copy
nexus_mem_free
nexus_mem_read
nexus_mem_set

Networking

Function
nexus_http_get

OS arguments

Function
nexus_os_args_count
nexus_os_args_get

Particles

Function
nexus_gfx_particle_alive

Function
nexus_gfx_particle_create
nexus_gfx_particle_destroy
nexus_gfx_particles_clear
nexus_gfx_particles_draw
nexus_gfx_particles_update

Physics

Function
nexus_physics_aabb
nexus_physics_circle

Printing

Function
nexus_print_bool
nexus_print_f64
nexus_print_i64
nexus_print_str

Safety

Function
nexus_bounds_check

Scenes

Function
nexus_scene_add

Function
nexus_scene_clear
nexus_scene_create
nexus_scene_destroy
nexus_scene_load
nexus_scene_save

Screen

Function
nexus_screen_begin
nexus_screen_clear
nexus_screen_end
nexus_screen_height
nexus_screen_present
nexus_screen_put
nexus_screen_set_title
nexus_screen_width

String conversion

Function
nexus_str_bool
nexus_str_concat
nexus_str_equal
nexus_str_f64
nexus_str_i64

Strings

Function
nexus_string_byte_at
nexus_string_contains
nexus_string_find
nexus_string_len
nexus_string_lower
nexus_string_slice
nexus_string_split_at
nexus_string_split_count
nexus_string_trim
nexus_string_upper

System

Function
nexus_beep
nexus_capture_output
nexus_clear
nexus_env
nexus_exit
nexus_key_pressed
nexus_random_i64
nexus_read_key
nexus_sleep_ms
nexus_system

Function
nexus_time_ms

Text and fonts

Function
nexus_gfx_font_default
nexus_gfx_font_load
nexus_gfx_font_style
nexus_gfx_font_unload
nexus_gfx_text
nexus_gfx_text_font
nexus_gfx_text_height
nexus_gfx_text_width
nexus_gfx_texture_draw
nexus_gfx_texture_draw_frame
nexus_gfx_texture_filter
nexus_gfx_texture_height
nexus_gfx_texture_load
nexus_gfx_texture_unload
nexus_gfx_texture_width

UI

Function
nexus_ui_button
nexus_ui_checkbox
nexus_ui_label

Function
nexus_ui_panel
nexus_ui_progress
nexus_ui_slider